

ENSF 380

TOPIC 20: OBJECT ORIENTED PROGRAMMING PART III

SOLID

S – Single Responsibility Principle (SRP)

O – Open/Closed Principle (OCP)

L – Liskov Substitution Principle (LSP)

I – Interface Segregation Principle (ISP)

D – Dependency Inversion Principle (DIP)

Single Responsibility Principle

01_SRP_Violation

Example

- A Report class should handle the content and structure of the Report, not printing or saving the report
- Printing and saving the report are handled by other classes

```
public class Main {  
    public static void main(String[] args) {  
        Report report = new Report("Monthly Report");  
        report.generateReport();  
        report.printReport();  
    }  
}
```

Single Responsibility Principle

01_SRP_Violation

```
public class Report {  
    public Report(String title, String content) {  
        this.title = title;  
        this.content = content;  
    }  
  
    public void generateReport() {  
        System.out.println("Generating Report Content...");  
        content = "Report Title: " + title + "\nReport Body: Sample content for the  
report.";  
    }  
  
    public void printReport() {  
        System.out.println("==== REPORT ====");  
        System.out.println(content);  
        System.out.println("=====");  
    }  
}
```

Single Responsibility Principle

02_SRP

```
public class Main {  
    public static void main(String[] args) {  
        // Create the report  
        Report report = new Report("Monthly Report");  
  
        // Generate content  
        ReportGenerator generator = new ReportGenerator();  
        generator.generate(report);  
  
        // Print the report  
        ReportPrinter printer = new ReportPrinter();  
        printer.print(report);  
    }  
}
```

Single Responsibility Principle

02_SRP

```
public class Report {  
    private String title;  
    private String content;  
  
    public Report(String title) {  
        this.title = title;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public String getContent() {  
        return content;  
    }  
  
    public void setContent(String content) {  
        this.content = content;  
    }  
}
```

...

Single Responsibility Principle

02_SRP

```
+ public class ReportGenerator {  
    public void generate(Report report) {  
        System.out.println("Generating Report Content...");  
        String content = "Report Title: " + report.getTitle() + "\nReport Body: Sample  
content for the report.";  
        report.setContent(content);  
    }  
}
```

```
public class ReportPrinter {  
    public void print(Report report) {  
        System.out.println("==== REPORT ====");  
        System.out.println(report.getContent());  
        System.out.println("=====");  
    }  
}
```

Open/Closed Principle

03_OCP_Violation

Example

- A class PaymentProcessor handles multiple different types of payment methods

```
public class PaymentProcessor {
    public void processPayment(String paymentMethod, double amount) {
        if (paymentMethod.equalsIgnoreCase("credit_card")) {
            System.out.printf("Processing credit card payment of $%.2f%n", amount);
            // Logic for credit card payment
        } else if (paymentMethod.equalsIgnoreCase("paypal")) {
            System.out.printf("Processing PayPal payment of $%.2f%n", amount);
            // Logic for PayPal payment
        } else if (paymentMethod.equalsIgnoreCase("bank_transfer")) {
            System.out.printf("Processing bank transfer payment of $%.2f%n", amount);
            // Logic for bank transfer payment
        } else {
            System.out.println("Unsupported payment method: " + paymentMethod);
        }
    }
}
```


Open/Closed Principle

04_OCP

```
public class Main {  
    public static void main(String[] args) {  
        PaymentProcessor processor = new PaymentProcessor();  
  
        // Using different payment methods  
        PaymentMethod creditCard = new CreditCardPayment();  
        PaymentMethod paypal = new PayPalPayment();  
        PaymentMethod bankTransfer = new BankTransferPayment();  
  
        processor.processPayment(creditCard, 150.0);  
        processor.processPayment(paypal, 75.5);  
        processor.processPayment(bankTransfer, 200.0);  
    }  
}
```

Open/Closed Principle

04_OCP

```
public class PaymentProcessor {  
    public void processPayment(PaymentMethod method, double amount) {  
        method.pay(amount);  
    }  
}
```

Open/Closed Principle

04_OCP

```
public class CreditCardPayment implements PaymentMethod {  
    @Override  
    public void pay(double amount) {  
        System.out.printf("Processing credit card payment of $%.2f%n", amount);  
        // Logic for credit card payment  
    }  
}
```

```
public interface PaymentMethod {  
    void pay(double amount);  
}
```

Liskov Substitution Principle

05_LSP_Violation

Example

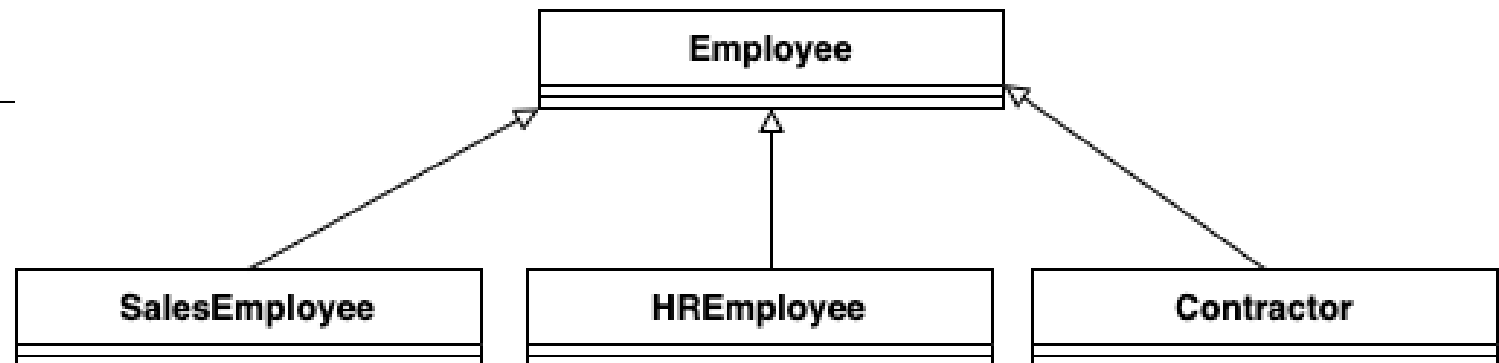
- A company has an Employee class which is inherited by different roles, such as Sales and HumanResources
- Each child class provides a different bonus calculation in the form of a calculateBonus() method
- The company also has Contractor which does not provide a calculateBonus() method

```
public abstract class Employee {  
    private String name;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
    public abstract double getBonus();  
}
```

Liskov Substitution Principle

05_LSP_Violation

```
public class SalesEmployee extends Employee {  
    public SalesEmployee(String name) {  
        super(name);  
    }  
  
    @Override  
    public double getBonus() {  
        return 2000.0;  
    }  
}
```



Liskov Substitution Principle

05_LSP_Violation

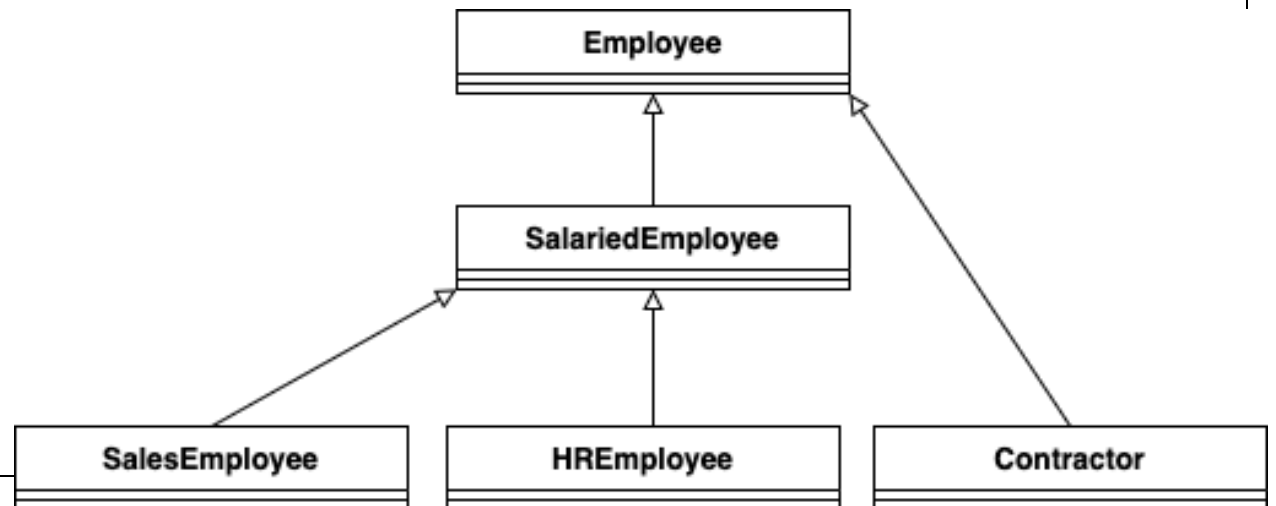
```
public class Contractor extends Employee {  
    public Contractor(String name) {  
        super(name);  
    }  
  
    @Override  
    public double getBonus() {  
        throw new UnsupportedOperationException("Contractors do not receive  
bonuses");  
    }  
}
```

Liskov Substitution Principle

06_LSP

```
public abstract class SalariedEmployee extends Employee {  
    public SalariedEmployee(String name) {  
        super(name);  
    }  
    public abstract double getBonus();  
}
```

```
public class SalesEmployee extends SalariedEmployee {  
    public SalesEmployee(String name) {  
        super(name);  
    }  
    @Override  
    public double getBonus() {  
        return 2000.0;  
    }  
}
```



Interface Segregation Principle

07_ISP_Violation

Example

- A company produces robot vacuums and robot mop-vacuums
- Instead of a Robot interface with methods for features such as vacuuming and mopping, this functionality should be split into smaller, specific interfaces so that each robot class only needs to implement the interfaces it needs

```
public interface CleaningRobot {  
    void vacuum();  
    void mop();  
}
```


Interface Segregation Principle

07_ISP_Violation

```
public class VacuumRobot implements CleaningRobot {  
    @Override  
    public void vacuum() {  
        System.out.println("Vacuuming the floor...");  
    }  
  
    @Override  
    public void mop() {  
        throw new UnsupportedOperationException("This robot cannot mop.");  
    }  
}
```

Interface Segregation Principle

08_ISP

```
public interface MopRobot {  
    void mop();  
}
```

```
public interface VacuumRobot {  
    void vacuum();  
}
```

```
... public class VacuumOnlyRobot implements VacuumRobot {
```

```
... public class MopVacuumRobot implements VacuumRobot, MopRobot {
```

Dependency Inversion Principle

09_DIP_Violation

Example

- If a Report class instantiates and uses a Database class, there will be a tight coupling between them

```
public class Report {  
    private PostgreSQLDatabase database;  
  
    public Report() {  
        this.database = new PostgreSQLDatabase();  
    }  
  
    public void generateReport() {  
        String data = database.retrieveSales();  
        System.out.println("Generating report with: " + data);  
    }  
}
```

Dependency Inversion Principle

10_DIP

```
public class Report {  
    private Database database;  
  
    public Report(Database database) {  
        this.database = database;  
    }  
  
    public void generateReport() {  
        String data = database.retrieveSales();  
        System.out.println("Generating report with: " + data);  
    }  
}
```

```
public interface Database {  
    String retrieveSales();  
}
```

Dependency Inversion Principle

10_DIP

```
public class PostgreSQLDatabase implements Database {  
    @Override  
    public String retrieveSales() {  
        return "Sales data from PostgreSQL";  
    }  
}
```

Dependency Inversion Principle

10_DIP

```
public class MockDatabase implements Database {  
    private String mockSalesData;  
  
    public void setMockSalesData(String mockData) {  
        this.mockSalesData = mockData;  
    }  
  
    @Override  
    public String retrieveSales() {  
        return this.mockSalesData;  
    }  
}
```

Summary: Knowledge

SOLID

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Summary: Skills

Creating object oriented designs incorporating SOLID principles